

ARYPush Notification Service

Technical Specification

Document version	1.1
SDK version	1.1.0 (Android, iOS, Flutter)
API surface	5 endpoints, project-scoped
Audience	Backend engineering, mobile engineering, QA
Date	21 September 2026

Contents

- 1. [Architectural & Configuration Standard](#)
- 2. [API Mapping & Reference Table](#)
- 3. [Endpoint Reference](#)
- 4. [Response Handling & Retry Semantics](#)
- 5. [When the SDK Calls Each Endpoint](#)
- 6. [Sending Notifications](#)
- 7. [SDK Integration — Android \(Kotlin\)](#)
- 8. [SDK Integration — iOS \(Swift\)](#)
- 9. [SDK Integration — Flutter \(Dart\)](#)
- 10. [Postman Collection \(v2.1\)](#)
- 11. [Appendix — Security, Versioning and Compatibility](#)

1. Architectural & Configuration Standard

ARYPush is a push notification SDK for native Android, native iOS and Flutter applications, backed by a project-scoped REST API. The SDK owns the device side of the push lifecycle — permission, token, token refresh, notification display, tap handling and synchronisation — and reports device state to the backend through **exactly five endpoints**.

1.1 Configuration model

The host application supplies four values when it initializes the SDK. Nothing is hardcoded in the SDK, so one build works against any environment.

Value	Purpose	Android / Flutter	iOS
Base URL	Root of the push API, e.g. <code>https://easypanel.host</code>	<code>baseUrl</code>	<code>baseURL</code>
Application ID	Public label identifying the app. Not a credential	<code>applicationId</code>	<code>applicationId</code>
Project ID	Scopes every request to a project	<code>projectId</code>	<code>projectId</code>
Auth token	Bearer token sent on every request	<code>authToken</code>	<code>authToken</code>

All four live in `PushBackendConfig`, passed as the `backend` of `ARYPushConfig`. iOS spells the base URL `baseURL` in keeping with Swift naming conventions; the meaning is identical.

The base URL is used exactly as the app passes it. The SDK adds no version segment — no `/v1`, nothing — and there is no API-version setting. Each endpoint path is joined straight onto it:

Base URL passed in	Register Device is sent to
<code>https://easypanel.host</code>	<code>https://easypanel.host/api/notifications/devices/register?projectId=...</code>
<code>https://easypanel.host/</code>	the same — a trailing slash is ignored
<code>https://easypanel.host/push</code>	<code>https://easypanel.host/push/api/notifications/devices/register?projectId=...</code>

So the base URL comes straight from the app's own build configuration, and a deployment that serves the API under a path prefix just includes that prefix in the base URL.

Omitting `backend` entirely is supported: push notifications are still received and displayed, and no request is ever made to a server.

1.2 Request conventions

Every request the SDK makes follows these rules.

Rule	Value
Authentication header	<code>Authorization: Bearer {{bearer_token}}</code> — a device key, e.g. <code>Authorization: Bearer ary_device_key_XXXXXXXXXXXXXXXXXXXXXXX</code>
Content type	<code>Content-Type: application/json</code> on requests with a body

Rule	Value
Project scope	<code>projectId={{project_id}}</code> query parameter on every request
Core identifier	<code>installationId</code> — a persistent UUID generated by the SDK on first launch
Transport	HTTPS. Plain HTTP is accepted only for local testing
Encoding	UTF-8 JSON request and response bodies

`projectId` is appended by the transport layer rather than by each call, so it cannot be forgotten on a new endpoint.

1.3 Additional headers

The SDK also sends these headers. They are informational: the backend may log or ignore them, but must not require any of them.

Header	Example	Use
<code>Accept</code>	<code>application/json</code>	Content negotiation
<code>X-Application-Id</code>	<code>wallet_android</code>	Same value as <code>applicationId</code>
<code>X-Installation-Id</code>	<code>8f14e45f-...</code>	Correlation and logging
<code>X-Platform</code>	<code>android</code> , <code>ios</code>	Triage
<code>X-SDK-Version</code>	<code>1.1.0</code>	Compatibility triage
<code>X-App-Version</code>	<code>5.2.0</code>	Compatibility triage
<code>X-Request-ID</code>	UUID	Stable across retries of one logical request — ideal for idempotency and log correlation

1.4 Identifiers

Three identifiers describe a device. Keeping them distinct is the backend's most important data modelling decision.

Identifier	Identifies	Changes when	Sent to the backend
<code>installationId</code>	One app on one device	Never — only on reinstall	Yes, in every relevant body
<code>Push token</code>	A delivery route (FCM or APNs)	Whenever the OS decides: reinstall, restore, data clear	Yes
User ID	A person	Login and logout	No — kept on the device only

A device whose token rotates keeps its `installationId`. The backend should key installation records on `installationId` and treat `token` as a mutable attribute of that record.

1.5 What the backend does not receive

The SDK has capabilities with no corresponding endpoint. They work fully on the device and are **never** sent to the server:

Capability	SDK API	Behaviour
User identity	<code>login(userId)</code> , <code>logout()</code>	Stored locally
Tags	<code>addTags(...)</code> , <code>removeTags(...)</code>	Stored locally
Topics	<code>subscribeToTopic(...)</code>	Android: works — FCM delivers to the topic from the device's own subscription. iOS: recorded on the device only. APNs has no topics, and with no topics endpoint the backend cannot fan out, so an iOS topic subscription does not affect delivery. Use segments instead
Engagement events	automatic <code>notification_received</code> / <code>notification_opened</code>	Not transmitted

Adding any of these to the server later is an additive change: see section 11.2.

2. API Mapping & Reference Table

#	Operation	Method	Path	Query	Success	Triggered by
1	Register Device	POST	/api/notifications/devices/register	projectId	200 / 201	First launch; any change to the installation
2	Device Token Update	PUT	/api/notifications/devices/update	projectId	200 / 204	Push token refresh
3	Subscribe / Unsubscribe	PUT	/api/notifications/devices/toggle	projectId	200 / 204	Notification permission change
4	Add Segment Subscriber	POST	/api/segments/{segmentId}/subscribers	projectId	200 / 201	App calls <code>subscribeToSegment(id)</code>
5	Segment List	GET	/api/segments/list	projectId	200	App calls <code>getSegments()</code>

2.1 Expected status codes

Status	Meaning	SDK behaviour
200 , 201 , 204	Success	Operation complete
400	Malformed request	Permanent — dropped, not retried
401	Missing or invalid bearer token	Permanent with a static <code>authToken</code> ; one refresh-and-retry with an <code>AuthProvider</code>
403	Token valid, access to project denied	Permanent — dropped
404	Unknown project, segment or path	Permanent — dropped
409	Conflict	Permanent — dropped
422	Validation failed (e.g. unknown <code>applicationId</code>)	Permanent — dropped
408 , 425 , 429	Timeout, too early, rate limited	Retried with backoff
5xx	Server error	Retried with backoff

The status code drives the SDK's retry policy. A transient failure returned as `4xx` is dropped permanently; a permanent failure returned as `5xx` is retried until the attempt budget is exhausted. See section 4.

2.2 Error body

Any of these shapes is understood. The message is logged by the SDK; it is never shown to users.

```
{ "error": { "code": "invalid_project", "message": "Project not found" } }
```

```
{ "code": "invalid_project", "message": "Project not found" }
```

3. Endpoint Reference

3.1 Register Device

```
POST /api/notifications/devices/register?projectId={{project_id}}
```

Creates or refreshes the installation record.

Request body

```
{
  "token": "dGhpcyBpcyBhIHRva2Vu...",
  "platform": "android",
  "applicationId": "wallet_android",
  "installationId": "8f14e45f-ea1e-4f7a-b2c1-5b2d7a1c9e33",
  "provider": "fcm",
  "appVersion": "5.2.0",
  "appBuild": "520",
  "sdkVersion": "1.1.0",
  "notificationsEnabled": true,
  "device": {
    "osVersion": "14",
    "deviceModel": "Google Pixel 8",
    "locale": "en-PK",
    "timezone": "Asia/Karachi"
  }
}
```

Field	Type	Required	Description
token	string null	No	FCM registration token or APNs device token. Absent or null before the OS has issued one
platform	string	Yes	android or ios
applicationId	string	Yes	Public application label
installationId	string (UUID)	Yes	Upsert key
provider	string	Yes	fcm or apns — which transport issued token
appVersion	string	No	Host app version name
appBuild	string	No	Host app build number
sdkVersion	string	Yes	ARYPush SDK version
notificationsEnabled	boolean	Yes	Whether the OS currently permits notifications
device	object	No	Absent entirely when the host app disabled device-info collection
device.osVersion	string	No	e.g. 14, 17.4

Field	Type	Required	Description
<code>device.deviceModel</code>	string	No	e.g. Google Pixel 8, iPhone16,1
<code>device.locale</code>	string	No	BCP-47, e.g. en-PK
<code>device.timezone</code>	string	No	IANA, e.g. Asia/Karachi

Backend requirements

- **Upsert on `installationId`.** The SDK re-sends Register whenever any field changes — including events that change nothing in this body, such as login — so repeated identical calls must not create duplicate records.
- **Treat a missing field and a `null` field identically.** Android sends `null`; iOS omits the key. Both mean "no value".
- **One live token per device.** A token can move between installations after a device restore. When a Register arrives carrying a token already held by a *different* installation, clear it from the older one, or that device receives every message twice.

Status	When
200 / 201	Registered or refreshed
400	Malformed body
401	Bearer token missing or invalid
422	Unknown <code>applicationId</code> , or a token the provider rejects

3.2 Device Token Update

```
PUT /api/notifications/devices/update?projectId={{project_id}}
```

Replaces the push token held for an installation.

Request body

```
{
  "installationId": "8f14e45f-ea1e-4f7a-b2c1-5b2d7a1c9e33",
  "newToken": "bmV3IHRva2Vu...",
  "platform": "android",
  "notificationsEnabled": true,
  "appVersion": "5.2.0"
}
```

Field	Type	Required	Description
installationId	string (UUID)	Yes	Installation whose token changed
newToken	string	Yes	The replacement token
platform	string	Yes	android or ios
notificationsEnabled	boolean	No	Current OS permission state
appVersion	string	No	Host app version name

Backend requirements

- Idempotent: applying the same newToken twice leaves the record unchanged.
- The SDK always sends a Register carrying the same new token **before** this call, so the installation is guaranteed to exist. Returning 404 for an unknown installationId is therefore safe, and the operation is dropped rather than retried.

Status	When
200 / 204	Updated
404	Unknown installationId
422	Token rejected by the provider

3.3 Subscribe / Unsubscribe

```
PUT /api/notifications/devices/toggle?projectId={{project_id}}
```

Records whether the operating system currently permits notifications for a device.

Request body

```
{
  "token": "dGhpcyBpcyBhIHRva2Vu...",
  "notificationsEnabled": false
}
```

Field	Type	Required	Description
token	string	Yes	Current push token — the device key for this endpoint
notificationsEnabled	boolean	Yes	true = subscribed, false = unsubscribed

Backend requirements

- Look the device up by token .
- The SDK never sends this before the device has a token. The Register sent when the token arrives already carries notificationsEnabled , so nothing is lost by that.
- Suppress sends to devices where this is false : it keeps delivery metrics honest and avoids paying to send messages that cannot be displayed.

The permission can change without the app being told — a user turning notifications off in system Settings produces no callback. The SDK samples the state every time the app comes to the foreground and sends this call when it differs from the last value reported.

Status	When
200 / 204	Recorded
404	Unknown token

3.4 Add Segment Subscriber

```
POST /api/segments/{segmentId}/subscribers?projectId={{project_id}}
```

Adds an installation to a segment.

Path parameter

Parameter	Description
<code>segmentId</code>	Segment identifier, as returned by Segment List. Percent-encoded by the SDK

Request body — the full installation payload, identical in shape to Register Device (section 3.1):

```
{
  "token": "dGhpcyBpcyBhIHRva2Vu...",
  "platform": "ios",
  "applicationId": "wallet_ios",
  "installationId": "3c9d2a7e-1f4b-4d8e-9a6c-7b2e5f1d0c84",
  "provider": "apns",
  "appVersion": "5.2.0",
  "appBuild": "520",
  "sdkVersion": "1.1.0",
  "notificationsEnabled": true,
  "device": {
    "osVersion": "17.4",
    "deviceModel": "iPhone16,1",
    "locale": "en-PK",
    "timezone": "Asia/Karachi"
  }
}
```

Backend requirements

- **Adding an existing subscriber should succeed with 2xx**. The SDK treats 409 as a failure and reports `false` to the application, which would present a device that *is* subscribed as having failed.
- Upsert the installation from the payload if it is not yet known.

This call is **not queued**. It is made only when the application calls `subscribeToSegment(segmentId)`, and its result is returned to the application directly, so it fails while offline rather than reporting an optimistic success.

Status	When
200 / 201	Subscribed, or already subscribed
404	Unknown segment
422	Invalid installation payload

3.5 Segment List

```
GET /api/segments/list?projectId={{project_id}}
```

Returns the segments defined for the project. No request body.

Response — any of these shapes is accepted:

```
[
  {
    "id": "seg_premium_pk",
    "name": "Premium Pakistan Users",
    "description": "subscription == premium AND country == PK",
    "joinedAt": 1730000000000
  }
]
```

```
{ "segments": [ { "id": "seg_premium_pk", "name": "Premium Pakistan Users" } ] }
```

```
{ "data": [ { "segmentId": "seg_premium_pk", "name": "Premium Pakistan Users" } ] }
```

Field	Type	Required	Description
id or segmentId	string	Yes	Entries without one are skipped
name	string	No	Defaults to the id
description	string	No	
joinedAt	number	No	Epoch milliseconds

This is the **project's segment catalogue**, not the segments a particular device belongs to. The SDK does not cache it: a call is made each time the application asks.

Status	When
200	Segment list, possibly empty

4. Response Handling & Retry Semantics

Registration, token updates and toggles are written to a **durable queue** on the device. They survive app restarts and airplane mode, and are delivered once connectivity returns. Segment subscription and the segment list are direct calls whose results go straight to the application.

4.1 Retry policy

Condition	Behaviour
2xx	Success
400 , 403 , 404 , 409 , 422	Permanent. The operation is dropped and never retried
401 with a static authToken	Permanent
401 with an AuthProvider	One token refresh, then exactly one retry. Never loops
408 , 425 , 429 , 5xx , network error	Retried with exponential backoff and full jitter
2xx whose body cannot be parsed	Permanent — a malformed success is a contract violation

Retry-After is honoured on retryable responses, in seconds or as an HTTP date, capped at 15 minutes so that a mistaken header cannot stall a device's queue.

4.2 Two rules for the backend

1. **Never return 4xx for a transient failure.** A database timeout, an unavailable dependency or a deployment in progress must be 5xx. A 4xx tells the SDK to give up permanently.
2. **Never return 5xx for a permanent rejection.** An unknown applicationId or a malformed body must be 4xx. A 5xx makes the SDK retry a request that can never succeed.

Both mistakes cause silent data loss: no error is visible anywhere.

4.3 Idempotency

Every queued request may be delivered **more than once**. Registration is an upsert keyed on installationId ; token updates and toggles are replacements. X-Request-ID is stable across all retries of one logical request and can be used to deduplicate.

5. When the SDK Calls Each Endpoint

Event on the device	Requests, in order
First launch, no token yet	1. Register (token absent)
First token issued	1. Register (with token) · 2. Token Update
Token refreshed by the OS	1. Register (new token) · 2. Token Update
User turns notifications off in Settings, reopens app	1. Register (notificationsEnabled: false) · 2. Toggle

Event on the device	Requests, in order
App updated to a new version	1. Register (new <code>appVersion</code>)
User logs in or out	1. Register (body unchanged)
Relaunch with nothing changed	<i>No request</i> — the SDK hashes the record and skips unchanged registrations
App calls <code>subscribeToSegment(id)</code>	Add Segment Subscriber
App calls <code>getSegments()</code>	Segment List

Register always precedes Token Update and Toggle for the same device state, so those two calls never arrive for an installation the backend has not been told about.

6. Sending Notifications

The backend holds the delivery credentials and sends through Firebase Cloud Messaging (Android, and iOS apps that use Firebase) or APNs (iOS). **No credential ever ships inside the mobile app.**

Platform	Credential	Where it lives
Android	FCM service account (HTTP v1 API)	Backend secret store
iOS	APNs authentication key (<code>.p8</code>) or certificate	Backend secret store

Route each message by the device's `provider`: an FCM token sent through APNs, or the reverse, reaches nobody.

6.1 Send data-only messages

A data-only message is the only shape that behaves identically in the foreground, in the background and when the app is terminated, on both platforms. A message with a `notification` block is displayed by the operating system before the SDK sees it, which loses deduplication, action buttons and tap tracking.

- **FCM:** send only the `data` object; omit `notification`.
- **APNs:** set `content-available: 1` and place the keys below in the payload.

6.2 Payload keys

```
{
  "ary_push": "1",
  "notification_id": "order-42",
  "title": "Order shipped",
  "body": "Your order is on its way",
  "image_url": "https://cdn.ary.com/order.png",
  "channel_id": "orders",
  "action": "open_order",
  "orderId": "12345",
  "actions": "[{\"id\":\"track\",\"title\":\"Track\"}]"
}
```

Key	Purpose
<code>ary_push</code>	Marks the message as ARYPush's, so an app with its own messaging code can tell them apart
<code>notification_id</code>	Stable message identity — always send it. Enables deduplication across resends
<code>title</code> , <code>body</code>	Notification content
<code>image_url</code>	Large image
<code>channel_id</code>	Android notification channel; falls back to the SDK default if unknown
<code>action</code> and custom keys	Passed verbatim to the app's tap handler, which decides where to navigate
<code>actions</code>	Action buttons, as a JSON-encoded string

The SDK never navigates on its own. The set of `action` values is a contract between the campaigns that send them and the app code that handles them — agree it with the mobile team and document it.

7. SDK Integration — Android (Kotlin)

Requirements: `minSdk` 21, `compileSdk` 36, AGP 9.0.1, Kotlin 2.3.20, JDK 17.

7.1 Dependency setup

`settings.gradle.kts` — the SDK is served by JitPack from the public repository, with no credentials:

```
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
        maven { url = uri("https://jitpack.io") }
    }
}
```

`app/build.gradle.kts`:

```
plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
    id("com.google.gms.google-services")
}

dependencies {
    // `main-SNAPSHOT` tracks the latest commit and works before any release is tagged.
    // Pin a release tag for production once one exists, e.g. `v1.1.0`.
    implementation("com.github.arysoftware:ary-push-sdk:main-SNAPSHOT")
}
```

Place your Firebase project's `google-services.json` in `app/`. The SDK ships no Firebase configuration of its own.

7.2 Initialization — `Application.onCreate`

```
package com.example.wallet

import android.app.Application
import com.ary.push.ARYPush
import com.ary.push.ARYPushConfig
import com.ary.push.PushBackendConfig
import com.ary.push.PushLogLevel

class WalletApplication : Application() {

    override fun onCreate() {
        super.onCreate()

        ARYPush.initialize(
            this,
            ARYPushConfig(
                enableLogging = true, // set to false for release builds
                logLevel = PushLogLevel.DEBUG,
                backend = PushBackendConfig(
                    baseUrl = "https://easypanel.host",
                    applicationId = "wallet_android",
                    projectId = "YOUR_PROJECT_ID",
                    authToken = "ary_device_key_XXXXXXXXXXXXXXXXXXXXXXX"
                )
            )
        )

        // Registered here rather than in an Activity: a tap that launched the app from a
        // terminated state is replayed to the first listener, and no Activity exists yet.
        ARYPush.addNotificationOpenedListener { notification ->
            NotificationRouter.route(this, notification)
        }
    }
}
```

Register the class in `AndroidManifest.xml`:

```
<application
    android:name=".WalletApplication"
    ... >
```

The SDK's messaging service, tap handler and permissions arrive through manifest merging — no other manifest entries are needed.

7.3 Notification click listener

```
package com.example.wallet

import android.content.Context
import android.content.Intent
import com.ary.push.model.PushNotification

object NotificationRouter {

    fun route(context: Context, notification: PushNotification) {
        val intent = when (notification.action) {
            "open_order" -> Intent(context, OrderActivity::class.java)
                .putExtra("orderId", notification.data["orderId"])
            else -> Intent(context, MainActivity::class.java)
        }
        context.startActivity(intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK))
    }
}
```

`notification.actionId` is set when the user tapped an action button rather than the notification itself.

7.4 Permission and segments

```
// Android 13+: ask at a moment the user understands why.
ARYPush.requestPermission { status -> /* update UI */ }

// Segment list and subscription.
ARYPush.getSegments { segments ->
    val premium = segments.firstOrNull { it.name == "Premium Pakistan Users" }
    if (premium != null) {
        ARYPush.subscribeToSegment(premium.id) { subscribed -> /* ... */ }
    }
}
```

Suspending forms of `getSegments()` and `subscribeToSegment(id)` are available inside coroutines.

7.5 Changing the token after login

Call `initialize` again with the new token; the SDK reconfigures in place and keeps its queue, listeners and installation id. `pushConfig` is the `ARYPushConfig` the app initialized with:

```
fun onUserSignedIn(context: Context, pushConfig: ARYPushConfig, newToken: String) {
    ARYPush.initialize(
        context,
        pushConfig.copyWith(backend = pushConfig.backend?.copy(authToken = newToken))
    )
}
```

For tokens that expire, supply an `AuthProvider` (`ARYPushConfig(authProvider = ...)`) instead. It is asked for a token on every request, can refresh once on `401`, and takes precedence over the static `authToken`.

8. SDK Integration — iOS (Swift)

Requirements: iOS 13, Xcode 15, Swift 5.9.

8.1 Swift Package Manager

1. In Xcode, choose **File > Add Package Dependencies...**
2. Enter `https://github.com/arysoftware/ary-push-sdk`
3. Set **Dependency Rule** to *Branch* `main` — or, once release `v1.1.0` is tagged, *Up to Next Major Version* from `1.1.0`
4. Add the **ARYPush** library to your app target.

The repository is public: no GitHub account needs to be signed in to Xcode. Or declare it in a `Package.swift`:

```
dependencies: [  
    // Swap for `from: "1.1.0"` once that release is tagged.  
    .package(url: "https://github.com/arysoftware/ary-push-sdk.git", branch: "main")  
],  
targets: [  
    .target(name: "YourApp", dependencies: [  
        .product(name: "ARYPush", package: "ary-push-sdk")  
    ])  
]
```

8.2 Capabilities — required

Under the app target's **Signing & Capabilities**:

1. **+ Capability > Push Notifications**. Without it iOS issues no device token at all.
2. **+ Capability > Background Modes**, then tick **Remote notifications** (for background and silent messages).

Test on a physical device: the iOS Simulator never receives an APNs device token.

8.3 Initialization — AppDelegate

```
import UIKit
import ARYPush

@main
final class AppDelegate: UIResponder, UIApplicationDelegate {

    var window: UIWindow?

    func application(
        _ application: UIApplication,
        didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey: Any]?
    ) -> Bool {

        ARYPush.initialize(
            ARYPushConfig(
                enableLogging: true,
                logLevel: .debug,
                backend: PushBackendConfig(
                    baseUrl: "https://easypanel.host",
                    applicationId: "wallet_ios",
                    projectId: "YOUR_PROJECT_ID",
                    authToken: "ary_device_key_XXXXXXXXXXXXXXXXXXXXXXXXXXXX"
                )
            )
        )

        // Registered during launch so a tap that launched the app from a terminated state is
        // replayed to this listener rather than lost.
        ARYPush.addNotificationOpenedListener { notification in
            NotificationRouter.route(notification)
        }

        return true
    }
}
```

No other `AppDelegate` code is needed. The SDK registers with APNs, receives the device token and installs its notification-centre delegate automatically. An existing `UNUserNotificationCenterDelegate` keeps working: the SDK forwards to it rather than replacing it.

8.4 Notification tap handling

```
import ARYPush

enum NotificationRouter {

    static func route(_ notification: PushNotification) {
        switch notification.action {
        case "open_order":
            AppNavigator.shared.openOrder(id: notification.data["orderId"])
        default:
            AppNavigator.shared.openHome()
        }
    }
}
```

The listener is called on the main queue. `notification.actionId` is set when an action button was tapped.

8.5 Permission and segments

```
ARYPush.requestPermission { status in
    // update UI
}

Task {
    let segments = await ARYPush.getSegments()
    if let premium = segments.first(where: { $0.name == "Premium Pakistan Users" }) {
        let subscribed = await ARYPush.subscribeToSegment(premium.id)
        print("Subscribed: \(subscribed)")
    }
}
```

8.6 Firebase Messaging on iOS

iOS gives the SDK an **APNs device token** (`provider: apns`). If your backend sends to iOS through Firebase instead, hand the SDK the FCM registration token so it registers that one:

```
func messaging(_ messaging: Messaging, didReceiveRegistrationToken fcmToken: String?) {
    if let fcmToken { ARYPush.setFCMToken(fcmToken) }
}
```

9. SDK Integration — Flutter (Dart)

Requirements: Flutter 3.10+, Dart 3.0+. The plugin brings the native Android and iOS SDKs with it: no Gradle, Podfile or `AppDelegate` changes are needed beyond the platform prerequisites below.

9.1 `pubspec.yaml`

```
dependencies:
  flutter:
    sdk: flutter
  ary_push:
    git:
      url: https://github.com/arysoftware/ary-push-sdk.git
      path: flutter
      # Tracks the default branch. Add `ref: v1.1.0` to pin a release once it is tagged.
```

```
flutter pub get
```

Platform prerequisites

Platform	Add
Android	<code>android/app/google-services.json</code> and the <code>com.google.gms.google-services</code> Gradle plugin
iOS	In <code>ios/Runner.xcworkspace</code> : Push Notifications capability, and Background Modes › Remote notifications

9.2 Initialization — `main()`

```
import 'package:ary_push/ary_push.dart';
import 'package:flutter/material.dart';

final GlobalKey<NavigatorState> navigatorKey = GlobalKey<NavigatorState>();

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await ARYPush.initialize(
    const ARYPushConfig(
      enableLogging: true,
      logLevel: PushLogLevel.debug,
      backend: PushBackendConfig(
        baseUrl: 'https://easypanel.host',
        applicationId: 'wallet_flutter',
        projectId: 'YOUR_PROJECT_ID',
        authToken: 'ary_device_key_XXXXXXXXXXXXXXXXXXXXXXXXX',
      ),
    ),
  );

  // Subscribed before runApp so a tap that launched the app from a terminated state is
  // replayed to this listener rather than lost.
  ARYPush.onNotificationOpened.listen(_routeNotification);

  runApp(MyApp(navigatorKey: navigatorKey));
}
```

9.3 ARYPush.onNotificationOpened listener

```
void _routeNotification(PushNotification notification) {
  final NavigatorState? navigator = navigatorKey.currentState;
  if (navigator == null) return;

  switch (notification.action) {
    case 'open_order':
      navigator.pushNamed('/order', arguments: notification.data['orderId']);
    default:
      navigator.pushNamed('/');
  }
}
```

9.4 Permission, token and segments

```
final PushPermissionStatus status = await ARYPush.requestPermission();

// The token arrives asynchronously -- on iOS after a round trip to Apple -- so listen rather
// than reading it once immediately after initialize().
ARYPush.onTokenRefresh.listen((String token) => debugPrint('Push token: $token'));

final List<Segment> segments = await ARYPush.getSegments();
final bool subscribed = await ARYPush.subscribeToSegment(segments.first.id);
```

9.5 Changing the token after login

Flutter has no `AuthProvider`; call `initialize` again with the new token. The SDK reconfigures in place and keeps its queue, listeners and installation id:

```
await ARYPush.initialize(
  ARYPushConfig(
    backend: PushBackendConfig(
      baseUrl: 'https://easypanel.host',
      applicationId: 'wallet_flutter',
      projectId: 'YOUR_PROJECT_ID',
      authToken: newToken,
    ),
  ),
);
```

9.6 iOS with Firebase Messaging

If your backend sends to iOS through Firebase, pass the FCM token on. It is ignored on Android, so this can live in shared code:

```
FirebaseMessaging.instance.onTokenRefresh.listen(ARYPush.setFCMToken);
final String? fcmToken = await FirebaseMessaging.instance.getToken();
if (fcmToken != null) await ARYPush.setFCMToken(fcmToken);
```

10. Postman Collection (v2.1)

Save the JSON below as `ARYPush.postman_collection.json` — the same file is in the repository at [postman/ARYPush.postman_collection.json](#) — and import it with **Postman › Import**.

Before sending a request, open the collection's **Variables** tab and set:

Variable	Example	Used in
<code>base_url</code>	<code>https://easypanel.host</code> — as-is, no version suffix	Every request
<code>bearer_token</code>	<i>(your device key, <code>ary_device_key_...</code>)</i>	Every request, via collection-level Bearer auth
<code>application_id</code>	<code>wallet_android</code>	Register, Add Segment Subscriber
<code>project_id</code>	<i>(your project id)</i>	Every request, as the <code>projectId</code> query parameter
<code>installation_id</code>	<code>8f14e45f-ea1e-4f7a-b2c1-5b2d7a1c9e33</code>	Register, Token Update, Add Segment Subscriber
<code>segment_id</code>	<i>(from Segment List)</i>	Add Segment Subscriber

Replace the `FCM_OR_APNS_DEVICE_TOKEN` placeholders in request bodies with a real token to test delivery end to end.

```

{
  "info": {
    "_postman_id": "6f2d8c1e-4b7a-4e1f-9a53-2c8e0b7d4a19",
    "name": "ARYPush Notification Service",
    "description": "The five endpoints the ARYPush SDK calls, exactly as the Android, iOS and Flutter clients send them.\n\nSet the collection variables before sending anything. Every request inherits `Authorization: Bearer {{bearer_token}}` from the collection and carries `projectId={{project_id}}` as a query parameter.\n\nThe `token` values in request bodies are placeholders: replace them with a real FCM registration token (Android) or APNs device token (iOS) to test delivery end to end.",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
  },
  "auth": {
    "type": "bearer",
    "bearer": [
      {
        "key": "token",
        "value": "{{bearer_token}}",
        "type": "string"
      }
    ]
  },
  "variable": [
    {
      "key": "base_url",
      "value": "https://easypanel.host",
      "type": "string"
    },
    {
      "key": "bearer_token",
      "value": "",
      "type": "string"
    },
    {
      "key": "application_id",
      "value": "wallet_android",
      "type": "string"
    },
    {
      "key": "project_id",
      "value": "",
      "type": "string"
    },
    {
      "key": "installation_id",
      "value": "8f14e45f-ea1e-4f7a-b2c1-5b2d7a1c9e33",
      "type": "string"
    },
    {
      "key": "segment_id",
      "value": "",
      "type": "string"
    }
  ],
  "item": [
    {
      "name": "1. Register Device",
      "request": {
        "method": "POST",
        "header": [
          {
            "key": "Content-Type",
            "value": "application/json"
          }
        ]
      }
    }
  ]
}

```

```

    ],
    "body": {
      "mode": "raw",
      "raw": "{\n  \"token\": \"FCM_OR_APNS_DEVICE_TOKEN\", \n  \"platform\": \"android\", \n\n  \"applicationId\": \"{{application_id}}\", \n  \"installationId\": \"{{installation_id}}\", \n\n  \"provider\": \"fcm\", \n  \"appVersion\": \"5.2.0\", \n  \"appBuild\": \"520\", \n  \"sdkVersion\": \"1.1.0\", \n  \"notificationsEnabled\": true, \n  \"device\": {\n    \"osVersion\": \"14\", \n    \"deviceModel\": \"Google Pixel 8\", \n    \"locale\": \"en-PK\", \n    \"timezone\": \"Asia/Karachi\" \n  }\n}",
      "options": {
        "raw": {
          "language": "json"
        }
      }
    },
    "url": {
      "raw": "{{base_url}}/api/notifications/devices/register?projectId={{project_id}}",
      "host": [
        "{{base_url}}"
      ],
      "path": [
        "api",
        "notifications",
        "devices",
        "register"
      ],
      "query": [
        {
          "key": "projectId",
          "value": "{{project_id}}"
        }
      ]
    },
    "description": "Creates or refreshes the installation. Sent on first launch and again whenever any field changes, so it must be an idempotent upsert keyed on `installationId`. \n\nOn iOS, set `platform` to `ios` and `provider` to `apns` (or `fcm` when the app supplies an FCM token). `device` is omitted entirely when the host app disabled device-info collection."
  },
  "response": []
},
{
  "name": "2. Device Token Update",
  "request": {
    "method": "PUT",
    "header": [
      {
        "key": "Content-Type",
        "value": "application/json"
      }
    ],
    "body": {
      "mode": "raw",
      "raw": "{\n  \"installationId\": \"{{installation_id}}\", \n  \"newToken\": \"NEW_FCM_OR_APNS_DEVICE_TOKEN\", \n  \"platform\": \"android\", \n  \"notificationsEnabled\": true, \n  \"appVersion\": \"5.2.0\" \n}",
      "options": {
        "raw": {
          "language": "json"
        }
      }
    }
  },
  "url": {
    "raw": "{{base_url}}/api/notifications/devices/update?projectId={{project_id}}",
    "host": [

```

```

        "{{base_url}}"
    ],
    "path": [
        "api",
        "notifications",
        "devices",
        "update"
    ],
    "query": [
        {
            "key": "projectId",
            "value": "{{project_id}}"
        }
    ]
},
"description": "Replaces the push token for an installation. Sent on every token refresh,
always after a Register that already carries the same new token."
},
"response": []
},
{
"name": "3. Subscribe / Unsubscribe",
"request": {
    "method": "PUT",
    "header": [
        {
            "key": "Content-Type",
            "value": "application/json"
        }
    ],
    "body": {
        "mode": "raw",
        "raw": "{\n  \"token\": \"FCM_OR_APNS_DEVICE_TOKEN\", \n  \"notificationsEnabled\":
false\n}",
        "options": {
            "raw": {
                "language": "json"
            }
        }
    }
},
"url": {
    "raw": "{{base_url}}/api/notifications/devices/toggle?projectId={{project_id}}",
    "host": [
        "{{base_url}}"
    ],
    "path": [
        "api",
        "notifications",
        "devices",
        "toggle"
    ],
    "query": [
        {
            "key": "projectId",
            "value": "{{project_id}}"
        }
    ]
},
"description": "Records whether the OS currently permits notifications for the device,
identified by its push token. Sent whenever the permission state changes, including when the user
turns notifications off in system Settings. Never sent before the device has a token."
},
"response": []
},

```

```

{
  "name": "4. Add Segment Subscriber",
  "request": {
    "method": "POST",
    "header": [
      {
        "key": "Content-Type",
        "value": "application/json"
      }
    ],
    "body": {
      "mode": "raw",
      "raw": "{\n  \"token\": \"FCM_OR_APNS_DEVICE_TOKEN\",\n  \"platform\": \"android\",\n  \"applicationId\": \"{{application_id}}\",\n  \"installationId\": \"{{installation_id}}\",\n  \"provider\": \"fcm\",\n  \"appVersion\": \"5.2.0\",\n  \"appBuild\": \"520\",\n  \"sdkVersion\": \"1.1.0\",\n  \"notificationsEnabled\": true,\n  \"device\": {\n    \"osVersion\": \"14\",\n    \"deviceModel\": \"Google Pixel 8\",\n    \"locale\": \"en-PK\",\n    \"timezone\": \"Asia/Karachi\"\n  }\n  \"options\": {\n    \"raw\": {\n      \"language\": \"json\"\n    }\n  }\n}",
      "options": {
        "raw": {
          "language": "json"
        }
      }
    },
    "url": {
      "raw": "{{base_url}}/api/segments/{{segment_id}}/subscribers?projectId={{project_id}}",
      "host": [
        "{{base_url}}"
      ],
      "path": [
        "api",
        "segments",
        "{{segment_id}}",
        "subscribers"
      ],
      "query": [
        {
          "key": "projectId",
          "value": "{{project_id}}"
        }
      ]
    },
    "description": "Adds the installation to a segment, sending the full installation payload. Sent only when the app calls subscribeToSegment(segmentId). Adding an installation that is already a subscriber should return 2xx: a 409 is treated by the SDK as a failure and reported to the app as `false`."
  },
  "response": []
},
{
  "name": "5. Segment List",
  "request": {
    "method": "GET",
    "header": [],
    "url": {
      "raw": "{{base_url}}/api/segments/list?projectId={{project_id}}",
      "host": [
        "{{base_url}}"
      ],
      "path": [
        "api",
        "segments",
        "list"
      ]
    }
  }
}

```

```
    "query": [
      {
        "key": "projectId",
        "value": "{{project_id}}"
      }
    ],
    "description": "Lists the project's segments. Sent only when the app calls getSegments();  
never cached. The SDK accepts a bare array, or an object wrapping one under `segments` or `data`.  
Each entry needs `id` (or `segmentId`); `name`, `description` and `joinedAt` (epoch ms) are  
optional."
  },
  "response": []
}
]
```

11. Appendix – Security, Versioning and Compatibility

11.1 Security

Rule	Detail
<code>applicationId</code> is not a credential	It is compiled into the app and readable by anyone. It must never authorize anything
The bearer token is never logged	The SDK excludes <code>Authorization</code> from logs at every level, and masks <code>authToken</code> when a configuration is printed
No token in <code>Info.plist</code>	iOS reads <code>ProjectId</code> from <code>Info.plist</code> but deliberately not the token: that file ships in the app as plain text
No delivery credentials in the app	FCM service accounts and APNs keys live only in the backend's secret store
TLS	Production traffic is HTTPS only. The SDK never disables certificate validation

A bearer token passed to the SDK is present in the device's memory for as long as the SDK runs. Issue a token scoped to device registration only, rather than reusing a user's full-access token.

11.2 Versioning

- Additive changes** – a new optional field, a new endpoint – are compatible with every SDK in the field.
- Breaking changes** – removing or renaming a field, changing a type, changing what a status code means, or moving a path – need a new path served alongside the old one.

A released mobile app cannot be made to update on the backend's schedule. Assume old SDK versions remain in use for at least a year, and that some devices never update.

11.3 SDK compatibility matrix

	Android	iOS	Flutter
Minimum OS	Android 5.0 (API 21)	iOS 13	per platform
Toolchain	AGP 9.0.1, Kotlin 2.3.20	Xcode 15, Swift 5.9	Flutter 3.10, Dart 3.0
Distribution	JitPack	Swift Package Manager, CocoaPods	<code>pubspec.yaml</code> git dependency
Transport	FCM	APNs (or FCM via <code>setFCMToken</code>)	FCM / APNs

11.4 Document history

Version	Date	Change
1.1	2026-09-21	Five project-scoped endpoints; <code>projectId</code> and <code>authToken</code> configuration; segment subscription
1.0	2026-09-03	Initial SDK release